BANCO DE DADOS

com Microsoft SQL Server

AULA 08

UPDATE · DELETE · Transações

Modificando dados com segurança no SISTEMA_VENDAS

UPDATE | UPDATE+JOIN | DELETE | TRUNCATE | BEGIN TRAN | COMMIT | ROLLBACK | TRY/CATCH

🎯 Objetivos da Aula 08

- ✏️ Atualizar dados com UPDATE — com e sem JOIN
- 🔐 Controlar transações com BEGIN TRANSACTION
- 🗑️ Excluir registros com DELETE de forma segura
- ✅ Confirmar alterações permanentemente com COMMIT
- ⚡ Entender TRUNCATE e quando usá-lo (vs DELETE)
- ↩️ Desfazer operações com ROLLBACK
- 📤 Usar OUTPUT para registrar o que foi alterado/excluído
- 🛡️ Tratar erros automaticamente com TRY/CATCH

🔄 Revisão — Aula 07: Funções SQL

❓ Diferença entre COUNT(*) e COUNT(coluna)?

✅ COUNT(*) conta todas as linhas; COUNT(coluna) ignora valores NULL

❓ Por que AVG pode distorcer resultados?

✅ Ignora NULL — divide só pelos não-nulos. Use AVG(ISNULL(col,0)) quando NULL=zero

❓ O que faz DATEDIFF(DAY, d1, d2)?

✅ Retorna a diferença em dias entre duas datas (d2 - d1)

❓ Diferença entre FORMAT e CONVERT?

✅ FORMAT é mais legível mas lento. CONVERT é mais rápido para grandes volumes

❓ O que faz NULLIF(expr, valor)?

✅ Retorna NULL se expr = valor — evita divisão por zero: 10/NULLIF(total,0)

⚠️ A Regra de Ouro do UPDATE e DELETE

🚨 **NUNCA execute UPDATE ou DELETE sem WHERE!**
Um único comando sem filtro pode alterar ou destruir TODOS os dados da tabela — sem aviso, sem confirmação.

Protocolo seguro — siga sempre esta ordem:

1 🔍 **Escreva o SELECT primeiro**

Substitua UPDATE/DELETE por SELECT * para ver exatamente o que será afetado

2 ✅ **Confirme o resultado**

Verifique: são esses os registros certos? A quantidade faz sentido?

3 🔐 **Envolva em transação**

BEGIN TRANSACTION antes de qualquer UPDATE/DELETE importante

4 ✏️ **Execute o UPDATE/DELETE**

Substitua SELECT por UPDATE ou DELETE — com o mesmo WHERE

5 ✅ **Verifique o resultado**

Execute SELECT novamente para confirmar que a alteração está correta

6 💾 **COMMIT ou ROLLBACK**

Se correto: COMMIT. Se algo errado: ROLLBACK e recomece

💻 UPDATE — Atualizando Dados com Segurança

```
USE SISTEMA_VENDAS;
-- PASSO 1: Sempre testar com SELECT antes!
SELECT id_cliente, nome, email FROM CLIENTES
WHERE  id_cliente = 1;  -- Verificar quem é
-- PASSO 2: Executar o UPDATE
UPDATE CLIENTES
SET    email   = 'ana.silva.novo@email.com',
       telefone = '11988887777'
WHERE  id_cliente = 1;
GO
-- PASSO 3: Confirmar a alteração
SELECT id_cliente, nome, email, telefone
FROM   CLIENTES WHERE id_cliente = 1;
-- Atualizar múltiplas colunas com expressão
UPDATE PRODUTOS
SET    preco   = preco * 1.10,    -- reajuste de 10%
       estoque = estoque + 50     -- reposição de estoque
WHERE  id_categoria = 1;          -- apenas Eletrônicos
GO
-- UPDATE com subquery no SET
UPDATE PEDIDOS
SET    total = (SELECT SUM(quantidade * preco_unitario)
                FROM   ITENS_PEDIDO ip
                WHERE  ip.id_pedido = PEDIDOS.id_pedido)
WHERE  total IS NULL OR total = 0;
GO  -- Recalcula total de todos os pedidos zerados
```

💻 UPDATE com OUTPUT — Registrando Alterações

```
-- OUTPUT em UPDATE: ver ANTES e DEPOIS da alteração
-- DELETED = valores ANTES do UPDATE
-- INSERTED = valores DEPOIS do UPDATE
-- Reajuste salarial com auditoria
UPDATE FUNCIONARIOS
SET    salario = salario * 1.08
OUTPUT DELETED.nome,
       DELETED.salario    AS salario_antes,
       INSERTED.salario   AS salario_depois,
       INSERTED.salario - DELETED.salario AS aumento
WHERE  cargo = 'Vendedor';
GO
-- Salvar auditoria em tabela de log
-- Criar tabela de log primeiro:
CREATE TABLE LOG_SALARIOS (
    id         INT IDENTITY PRIMARY KEY,
    nome       VARCHAR(150),
    sal_antes  DECIMAL(10,2),
    sal_depois DECIMAL(10,2),
    alterado_em DATETIME2 DEFAULT GETDATE()
); GO

-- UPDATE com OUTPUT INTO (salva no log automaticamente!)
UPDATE FUNCIONARIOS SET salario = salario * 1.05
OUTPUT DELETED.nome, DELETED.salario, INSERTED.salario
INTO   LOG_SALARIOS (nome, sal_antes, sal_depois)
```

💻 UPDATE com JOIN — Atualizando com Dados de Outra Tabela

```
-- UPDATE com JOIN: atualiza usando dados de outra tabela
-- Sintaxe SQL Server: UPDATE alias SET ... FROM tabela JOIN ...
-- Cenário: aplicar desconto de 5% em produtos de Alimentos
-- (a categoria está em outra tabela)
UPDATE p
SET    p.preco = p.preco * 0.95
FROM   PRODUTOS p
INNER  JOIN CATEGORIAS cat ON p.id_categoria = cat.id_categoria
WHERE  cat.nome_categoria = 'Alimentos';
GO
-- Verificar resultado:
SELECT p.nome_produto, p.preco, cat.nome_categoria
FROM   PRODUTOS p JOIN CATEGORIAS cat ON p.id_categoria=cat.id_categoria
WHERE  cat.nome_categoria = 'Alimentos';
-- Cenário 2: marcar pedidos como ENTREGUE se todos os itens existem
UPDATE p
SET    p.status = 'ENTREGUE'
FROM   PEDIDOS p
WHERE  p.status = 'ENVIADO'
AND    EXISTS (SELECT 1 FROM ITENS_PEDIDO ip
               WHERE  ip.id_pedido = p.id_pedido);
GO
-- Verificar pedidos atualizados:
SELECT id_pedido, status FROM PEDIDOS WHERE status = 'ENTREGUE';
```

💻 DELETE — Excluindo Registros com Segurança

```
-- REGRA: sempre testar com SELECT antes do DELETE!

-- PASSO 1: Ver o que será excluído
SELECT id_pedido, status, total FROM PEDIDOS
WHERE  status = 'CANCELADO';

-- PASSO 2: Excluir os itens primeiro (FK!)
DELETE FROM ITENS_PEDIDO
WHERE  id_pedido IN (SELECT id_pedido FROM PEDIDOS
                     WHERE  status = 'CANCELADO');
GO

-- PASSO 3: Excluir os pedidos cancelados
DELETE FROM PEDIDOS
WHERE  status = 'CANCELADO';
GO

-- DELETE com OUTPUT: registrar o que foi excluído
DELETE FROM CLIENTES
OUTPUT DELETED.id_cliente, DELETED.nome, DELETED.email,
       GETDATE() AS excluido_em
WHERE  id_cliente = 99;  -- cliente inexistente, zero linhas excluídas
GO

-- DELETE com TOP: excluir N registros por vez (lotes)
DELETE TOP(100) FROM ITENS_PEDIDO
WHERE  id_pedido IN (SELECT id_pedido FROM PEDIDOS
                     WHERE YEAR(data_pedido) < 2020);
-- Excluir em lotes evita lock na tabela inteira!
```

TRUNCATE vs DELETE — Diferenças que Importam

Característica	DELETE	TRUNCATE
O que remove?	Registros específicos (com WHERE) ou todos	TODOS os registros (sem WHERE)
Filtra com WHERE?	✅ Sim	❌ Não
IDENTITY reseta?	❌ Não	✅ Sim (volta ao seed)
Triggers disparam?	✅ Sim (por registro)	❌ Não
ROLLBACK possível?	✅ Sim (DML)	⚠️ Depende[1]
Performance	🐢 Mais lenta (log por linha)	⚡ Muito mais rápida
Respeita FK?	✅ Sim — erro se tiver filhos	✅ Sim — erro se tiver filhos
Permissão mínima?	DELETE na tabela	ALTER TABLE

[1] TRUNCATE pode ser revertido com ROLLBACK se estiver dentro de uma transação explícita (BEGIN TRANSACTION). Fora de transação, é permanente.

Transações — Garantindo a Integridade com ACID

Uma transação é um conjunto de operações SQL que formam uma UNIDADE LÓGICA de trabalho: ou todas são executadas com sucesso, ou nenhuma é aplicada. Isso garante consistência dos dados.

A — Atomicidade

Tudo ou nada. Se qualquer operação da transação falhar, TODAS são revertidas. Não existe execução parcial.

C — Consistência

O banco passa de um estado válido para outro estado válido. Constraints e regras são respeitadas durante toda a transação.

I — Isolamento

Transações concorrentes não se interferem. Cada transação 'enxerga' um snapshot consistente dos dados.

D — Durabilidade

Uma vez confirmada (COMMIT), a transação é permanente — mesmo em caso de falha de energia ou crash do servidor.

💡 **Analogia: Transferência bancária = débitar conta A + creditar conta B. Se o crédito falhar após o débito, o dinheiro 'desaparece'. Com transação: ou ambos acontecem, ou nenhum.**

💻 BEGIN TRANSACTION · COMMIT · ROLLBACK

```
-- Estrutura básica de uma transação
BEGIN TRANSACTION;  -- Inicia a transação

    -- Operações dentro da transação
    UPDATE FUNCIONARIOS
    SET    salario = salario * 1.10  -- reajuste 10%
    WHERE  cargo = 'Vendedor';

    -- Verificar o resultado ANTES de confirmar
    SELECT nome, salario FROM FUNCIONARIOS WHERE cargo = 'Vendedor';

   -- Se o resultado estiver correto:
   COMMIT;  -- Confirma permanentemente

   -- Se algo estiver errado:
   -- ROLLBACK;  -- Desfaz tudo desde o BEGIN TRANSACTION

   -- Transação nomeada (mais legível em scripts complexos)
   BEGIN TRANSACTION reajuste_salarios;
       UPDATE FUNCIONARIOS SET salario = salario * 1.05;
       -- verificações...
   COMMIT TRANSACTION reajuste_salarios;

   -- SAVEPOINT: ponto de salvamento parcial
   BEGIN TRANSACTION;
       UPDATE CLIENTES SET cidade = 'SP' WHERE id_cliente=1;
       SAVE TRANSACTION ponto1;
       UPDATE PRODUTOS SET preco = preco * 1.1;
       ROLLBACK TRANSACTION ponto1;  -- desfaz só o UPDATE de produtos
   COMMIT;  -- confirma só o UPDATE de clientes
```

💻 TRY/CATCH — Tratamento Automático de Erros

```sql
-- TRY/CATCH: captura erros e faz ROLLBACK automaticamente

BEGIN TRY
    BEGIN TRANSACTION;

    -- Atualizar status do pedido
    UPDATE PEDIDOS
    SET    status = 'ENVIADO'
    WHERE  id_pedido = 2 AND status = 'APROVADO';

    -- Reduzir estoque do produto
    UPDATE PRODUTOS
    SET    estoque = estoque - 1
    WHERE  id_produto = 1 AND estoque > 0;

    -- Verificar se o estoque ficou negativo (simulando validação)
    IF (SELECT estoque FROM PRODUTOS WHERE id_produto=1) < 0
        THROW 50001, 'Estoque insuficiente!', 1;

    COMMIT;
    PRINT 'Transação concluída com sucesso!  ✅';
END TRY
BEGIN CATCH
    ROLLBACK;  -- Desfaz tudo automaticamente!
    PRINT 'ERRO: ' + ERROR_MESSAGE();
    PRINT 'Linha: ' + CAST(ERROR_LINE() AS VARCHAR);
    PRINT 'Número: ' + CAST(ERROR_NUMBER() AS VARCHAR);
END CATCH
```

💻 Cenário Completo — Processando um Pedido

```
-- Cenário: Aprovar pedido + baixar estoque + registrar log
DECLARE @id_pedido   INT = 3;
DECLARE @id_produto  INT;
DECLARE @qtd         INT;
BEGIN TRY
    BEGIN TRANSACTION;
    -- 1. Verificar se o pedido existe e está pendente
    IF NOT EXISTS (SELECT 1 FROM PEDIDOS
                   WHERE id_pedido=@id_pedido AND status='PENDENTE')
        THROW 50002, 'Pedido não encontrado ou não está PENDENTE', 1;
    -- 2. Baixar estoque de cada produto do pedido
    UPDATE pr
    SET    pr.estoque = pr.estoque - ip.quantidade
    FROM   PRODUTOS pr
    JOIN   ITENS_PEDIDO ip ON pr.id_produto = ip.id_produto
    WHERE  ip.id_pedido = @id_pedido;
    -- 3. Verificar se algum estoque ficou negativo
    IF EXISTS (SELECT 1 FROM PRODUTOS WHERE estoque < 0)
        THROW 50003, 'Estoque insuficiente para um ou mais produtos', 1;
    -- 4. Aprovar o pedido
    UPDATE PEDIDOS SET status='APROVADO' WHERE id_pedido=@id_pedido;
    COMMIT;
    PRINT 'Pedido #' + CAST(@id_pedido AS VARCHAR) + ' aprovado!  ✅';
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT 'ERRO ao processar pedido: ' + ERROR_MESSAGE();
END CATCH
```

✏️ Exercícios Propostos — Aula 08

01 — Básico

Aplique um reajuste de 12% no preço de todos os produtos da categoria 'Roupas'. Use OUTPUT para ver os preços antes e depois. Confirme com SELECT.

02 — Básico

Delete todos os pedidos com status 'CANCELADO'. ATENÇÃO: exclua primeiro os itens em ITENS_PEDIDO (FK!), depois os pedidos. Use SELECT antes de cada DELETE.

03 — Médio

Crie uma transação que: (1) atualize o status do pedido 4 para APROVADO, (2) reduza o estoque do produto do pedido 4. Se o estoque ficar negativo, faça ROLLBACK. Use TRY/CATCH.

04 — Médio

Use UPDATE com subquery no SET para recalcular o campo 'total' de TODOS os pedidos com base na soma real de (quantidade × preco_unitario) de ITENS_PEDIDO.

05 — Avançado

Crie um script de 'encerramento do mês': BEGIN TRAN → (1) mover pedidos ENTREGUES há mais de 30 dias para tabela PEDIDOS_HISTORICO (SELECT INTO) → (2) deletar esses pedidos de PEDIDOS → (3) atualizar estatística na tabela FUNCIONARIOS. TRY/CATCH com ROLLBACK completo.

✅ Respostas Comentadas — Exercícios 03 e 04

```sql
-- EX 03: Aprovação de pedido com TRY/CATCH
BEGIN TRY
    BEGIN TRANSACTION;
    UPDATE PEDIDOS SET status='APROVADO' WHERE id_pedido=4;
    UPDATE pr SET pr.estoque = pr.estoque - ip.quantidade
    FROM PRODUTOS pr
    JOIN ITENS_PEDIDO ip ON pr.id_produto = ip.id_produto
    WHERE ip.id_pedido = 4;
    IF EXISTS (SELECT 1 FROM PRODUTOS WHERE estoque < 0)
        THROW 50003, 'Estoque ficou negativo!', 1;
    COMMIT; PRINT 'Pedido 4 aprovado!  ✅';
END TRY
BEGIN CATCH
    ROLLBACK; PRINT 'ERRO: ' + ERROR_MESSAGE();
END CATCH
-- EX 04: Recalcular total de todos os pedidos
UPDATE p
SET    p.total = (
    SELECT SUM(ip.quantidade * ip.preco_unitario)
    FROM   ITENS_PEDIDO ip
    WHERE  ip.id_pedido = p.id_pedido
)
FROM   PEDIDOS p;
GO
-- Verificar: totais agora correspondem à soma real dos itens
SELECT id_pedido, total FROM PEDIDOS ORDER BY id_pedido;
```

🏆 Desafio da Aula 08

📋 Contexto: Fim de trimestre no SISTEMA_VENDAS. O gerente precisa de um script de 'fechamento' que aplique reajustes, processe pendências, arquive histórico e gere log — tudo em transações seguras.

🎯 Scripts obrigatórios:

T1 Reajuste de preços: +8% em Eletrônicos, +5% em Roupas, -3% em Alimentos. Use UPDATE+JOIN por categoria. OUTPUT para ver antes/depois. Tudo em uma transação.

T2 Recalcular totais: atualizar campo 'total' de TODOS os pedidos com subquery (SUM de itens). Verificar pedidos sem itens (total = NULL) e marcar como 'REVISÃO'.

T3 Processar pendentes: para cada pedido PENDENTE há mais de 7 dias, tentar aprovar (baixar estoque). Se estoque insuficiente: ROLLBACK só desse pedido e marcar como 'SUSPENSO'.

T4 Limpar cancelados: salvar pedidos CANCELADOS em PEDIDOS_ARQUIVO (SELECT INTO) com data de arquivamento. Depois, DELETE em cascata (itens → pedidos). TRY/CATCH completo.

T5 BÔNUS: Criar uma stored procedure sp_fechar_trimestre que encapsula T1+T2+T3+T4 em sequência com uma única transação master e log de cada etapa.

📚 Resumo — Aula 08: UPDATE, DELETE e Transações

✏️ UPDATE

SET coluna=valor WHERE filtro. SEMPRE com WHERE. Testar com SELECT antes!

🗑️ DELETE

DELETE FROM tabela WHERE filtro. Respeitar ordem das FKs (filha→pai)

⚡ TRUNCATE

Remove tudo, reseta IDENTITY, sem log por linha. Revertível em transação!

📤 OUTPUT

DELETED.col e INSERTED.col — ver antes/depois. OUTPUT INTO salva em tabela

🔐 BEGIN TRANSACTION

Inicia unidade atômica. Operações ficam 'pendentes' até COMMIT ou ROLLBACK

✅ COMMIT

Confirma permanentemente. Só após COMMIT as outras sessões enxergam as mudanças

↩️ ROLLBACK

Desfaz tudo desde o BEGIN TRANSACTION. ROLLBACK TO savepoint reverte parcialmente

🛡️ TRY/CATCH

Captura qualquer erro e faz ROLLBACK automaticamente. Padrão profissional!

🚀 Próxima Aula 09: Ordenação e Agrupamento

Aula 09 — O que vem por aí:

⬆️ **ORDER BY avançado**	Múltiplas colunas, expressões e NULLS FIRST/LAST
📊 **GROUP BY**	Agrupar linhas com o mesmo valor — base de relatórios
🔍 **HAVING**	Filtrar GRUPOS (como WHERE filtra linhas)
📈 **Relatórios gerenciais**	Faturamento por período, por categoria, por vendedor
🔢 **Funções de janela**	ROW_NUMBER, RANK, DENSE_RANK — preview
📋 **ROLLUP e CUBE**	Subtotais automáticos para relatórios hierárquicos

📝 **Para a próxima aula: conclua o desafio de fechamento trimestral e garanta que os dados do SISTEMA_VENDAS estão consistentes.**